

**LAPORAN  
WORKSHOP SISTEM INFORMASI WEB SERVER  
(TIF320803)  
MINGGU 6 ACARA 15  
SEMESTER 3**



**Tugas :  
Konsep API dan JSON**

**Disusun Oleh:  
Uaryl Engno Laurenca ( E41250317)**

**PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)  
TEKNIK INFORMATIKA KAB. NGANJUK  
JURUSAN TEKNOLOGI INFORMASI  
POLITEKNIK NEGERI JEMBER  
TAHUN 2026**

## A. PENDAHULUAN

Pada praktikum-praktikum sebelumnya, aplikasi Sistem Informasi Akademik hanya dapat diakses melalui halaman web (View) yang dirender langsung oleh server. Namun dalam pengembangan aplikasi modern, data sering dibutuhkan oleh aplikasi lain seperti aplikasi mobile atau sistem pihak ketiga tanpa harus mengakses tampilan HTML. Untuk kebutuhan ini digunakan API (Application Programming Interface) sebagai jalur komunikasi antar aplikasi, dengan JSON (JavaScript Object Notation) sebagai format data yang dikirim dan diterima.

Pada Acara 15 ini, dibuat endpoint API sederhana pada `public/api/mahasiswa.php` yang memanfaatkan kembali (reuse) `MahasiswaService` dan `MahasiswaRepository` yang sudah dibangun pada acara-acara sebelumnya, sehingga API ini tetap konsisten dengan validasi dan logika bisnis yang sama dengan aplikasi web-nya.

## B. TUJUAN PRAKTIKUM

1. Mahasiswa mampu menjelaskan konsep API dan perannya dalam komunikasi antar aplikasi.
2. Mahasiswa mampu menjelaskan konsep JSON sebagai format data pertukaran.
3. Mahasiswa mampu membuat endpoint API sederhana menggunakan PHP dan PDO.
4. Mahasiswa mampu mengubah data dari MySQL menjadi response JSON menggunakan `json_encode()`.
5. Mahasiswa mampu menguji endpoint API menggunakan browser dan Postman.

## C. DASAR TEORI

API (Application Programming Interface) adalah mekanisme yang memungkinkan satu aplikasi berkomunikasi dengan aplikasi atau sistem lain. Frontend tidak mengakses database secara langsung, melainkan meminta data melalui API menggunakan protokol HTTP, dengan method seperti GET (mengambil data), POST (menambah data), PUT/PATCH (mengubah data), dan DELETE (menghapus data).

JSON (JavaScript Object Notation) adalah format pertukaran data yang ringan dan mudah dibaca, baik oleh manusia maupun komputer. JSON terdiri dari pasangan *key* → *value*, menggunakan `{ }` untuk object tunggal dan `[ ]` untuk kumpulan data (array of objects).

Hubungan API dan JSON. API dan JSON adalah dua hal yang berbeda: API merupakan *jalan komunikasi* antar aplikasi, sedangkan JSON adalah salah satu *format data* yang digunakan dalam komunikasi tersebut. Di PHP, array dapat diubah menjadi JSON menggunakan `json_encode()`, dan sebaliknya JSON dapat diubah menjadi array PHP menggunakan `json_decode()`.

## D. HASIL DAN PEMBAHASAN

### 1. Endpoint API

```
1 <?php
2
3 require __DIR__ . '/../config/app.php';
4 require __DIR__ . '/../app/Core/Logger.php';
5 require __DIR__ . '/../app/Core/Database.php';
6 require __DIR__ . '/../app/Models/Mahasiswa.php';
7 require __DIR__ . '/../app/Repositories/MahasiswaRepository.php';
8 require __DIR__ . '/../app/Repositories/ProdiRepository.php';
9 require __DIR__ . '/../app/Services/MahasiswaService.php';
10
11 use App\Core\Database;
12 use App\Repositories\MahasiswaRepository;
13 use App\Repositories\ProdiRepository;
14 use App\Services\MahasiswaService;
15
16 header('Content-Type: application/json');
17
18 try {
19     $db = Database::getInstance();
20     $service = new MahasiswaService(
21         new MahasiswaRepository($db),
22         new ProdiRepository($db)
23     );
24
25     $method = $_SERVER['REQUEST_METHOD'];
26
27     if ($method === 'GET') {
28         if (isset($_GET['id'])) {
29             $id = (int)$_GET['id'];
30             $mhs = $service->find($id);
31
32             if (!$mhs) {
33                 http_response_code(404);
34                 echo json_encode([
35                     'success' => false,
36                     'message' => 'Data mahasiswa tidak ditemukan',
37                 ]);
38                 exit;
39             }
40
41             echo json_encode([
42                 'success' => true,
43                 'message' => 'Data berhasil diambil',
44                 'data' => [
45                     'id' => $mhs->getId(),
46                     'nim' => $mhs->getNim(),
47                     'nama' => $mhs->getNama(),
48                     'email' => $mhs->getEmail(),
49                     'prodi_id' => $mhs->getProdiId(),
50                     'angkatan' => $mhs->getAngkatan(),
51                 ],
52             ]);
53             exit;
54         }
55
56         $list = $service->list();
57         $data = array_map(fn($m) => [
58             'id' => $m->getId(),
59             'nim' => $m->getNim(),
60             'nama' => $m->getNama(),
61             'email' => $m->getEmail(),
62             'prodi_id' => $m->getProdiId(),
63             'angkatan' => $m->getAngkatan(),
64         ], $list);
65
66         echo json_encode([
67             'success' => true,
68             'message' => 'Data berhasil diambil',
69             'data' => $data,
70         ]);
71         exit;
72     }
73
74     if ($method === 'POST') {
75         $input = json_decode(file_get_contents('php://input'), true) ?? [];
76
77         $result = $service->create($input);
78
79         http_response_code($result['success'] ? 201 : 400);
80         echo json_encode([
81             'success' => $result['success'],
82             'message' => $result['message'],
83         ]);
84         exit;
85     }
86
87     http_response_code(405);
88     echo json_encode([
89         'success' => false,
90         'message' => 'Method tidak didukung',
91     ]);
92 } catch (\Throwable $e) {
93     http_response_code(500);
94     echo json_encode([
95         'success' => false,
96         'message' => 'Terjadi kesalahan pada server',
97     ]);
98 }
99 }
```

Dibuat file `public/api/mahasiswa.php` yang menangani request HTTP dengan dua method.

- GET `/api/mahasiswa`, mengambil seluruh data mahasiswa.
- GET `/api/mahasiswa?id=1`, mengambil satu data mahasiswa berdasarkan ID.
- POST `/api/mahasiswa`, menambahkan data mahasiswa baru melalui body JSON.

Endpoint ini tidak membuat ulang logika akses database, melainkan memanggil ulang `MahasiswaService` yang sudah dibangun pada Acara 13–14, sehingga validasi NIM, nama, email, dan prodi tetap konsisten antara aplikasi web dan API.

## 2. Response Terstruktur

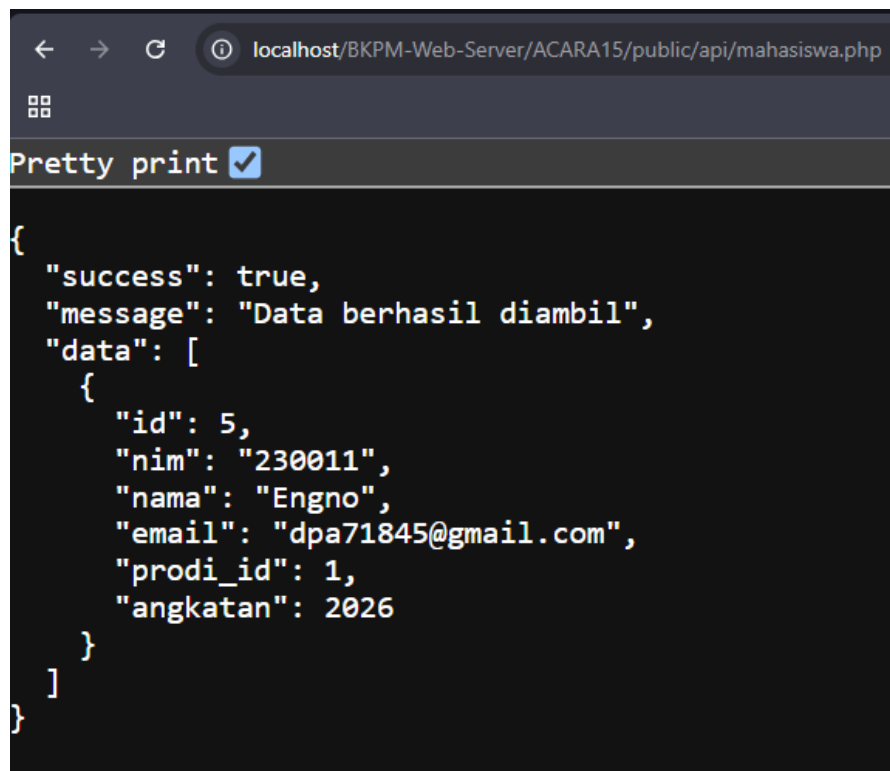
Setiap response API mengikuti format standar:

json

```
{
  "success": true,
  "message": "Data berhasil diambil",
  "data": []
}
```

Dengan format ini, aplikasi frontend dapat dengan mudah mengecek status keberhasilan (success) sebelum memproses isi data.

## 3. Pengujian melalui Browser

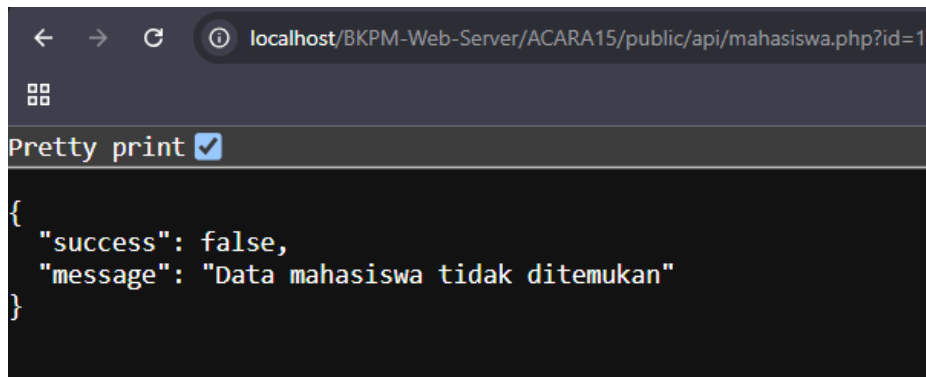


Endpoint diuji langsung melalui browser dengan mengakses:

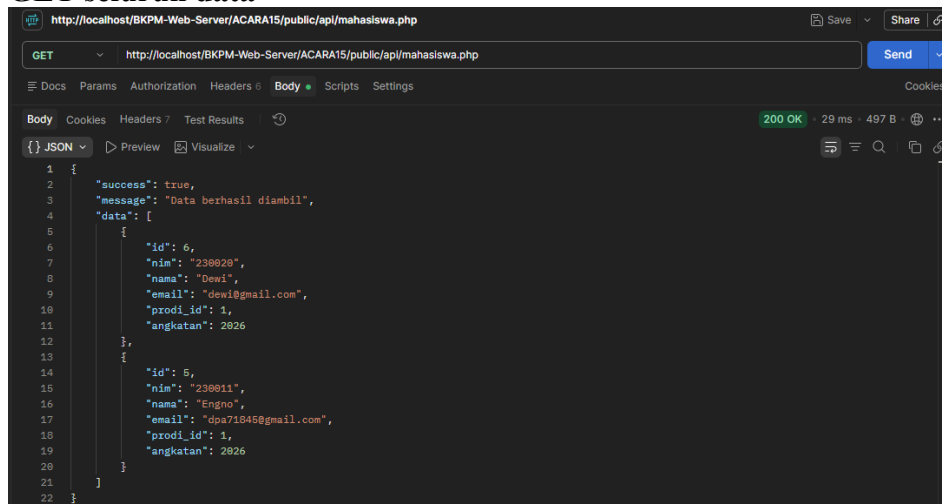
`http://localhost/BKPM-Web-Server/ACARA15/public/api/mahasiswa.php`

Serta pengujian ambil satu data berdasarkan ID:

`http://localhost/BKPM-Web-Server/ACARA15/public/api/mahasiswa.php?id=1`

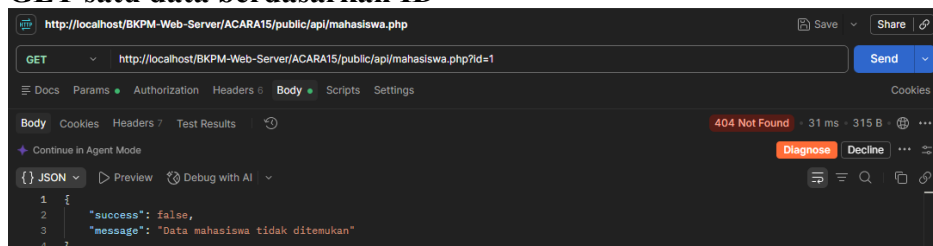


#### 4. Pengujian melalui Postman GET seluruh data



- Method: GET
- URL: <http://localhost/BKPM-Web-Server/ACARA15/public/api/mahasiswa.php>
- Hasil: status 200 OK, response berisi array seluruh data mahasiswa.

#### GET satu data berdasarkan ID



- Method: GET
- URL: <http://localhost/BKPM-Web-Server/ACARA15/public/api/mahasiswa.php?id=5>
- Hasil: status 200 OK jika data ditemukan, 404 Not Found jika ID tidak ada.

#### POST tambah data

- Method: POST
- URL: <http://localhost/BKPM-Web-Server/ACARA15/public/api/mahasiswa.php>
- Body → raw → JSON:

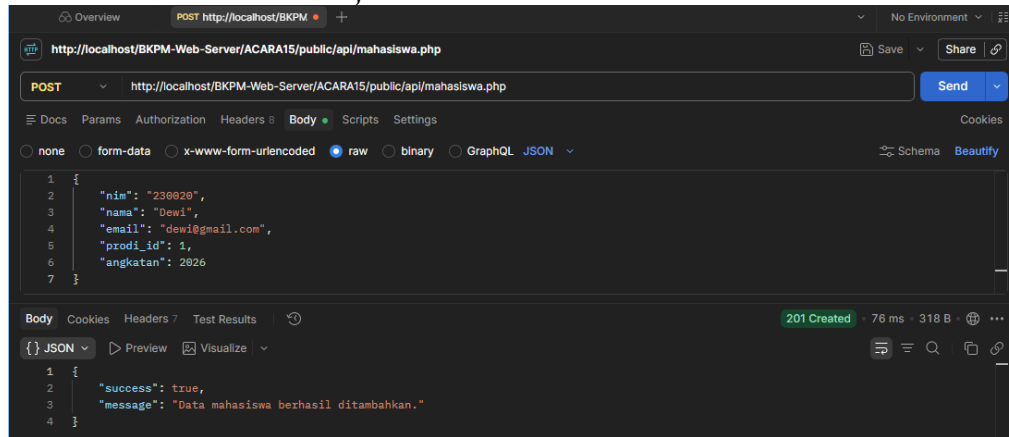
```
json
{
  "nim": "230020",
```

```

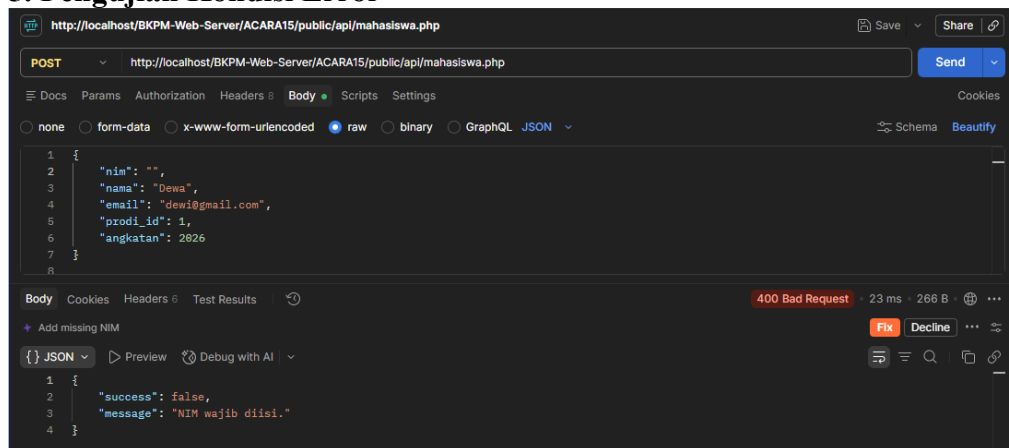
"nama": "Dewi",
"email": "dewi@gmail.com",
"prodi_id": 1,
"angkatan": 2026
}

```

- Hasil: status 201 Created, response {"success": true, "message": "Data mahasiswa berhasil ditambahkan."}.



## 5. Pengujian Kondisi Error



Dilakukan juga pengujian POST dengan data tidak lengkap (misalnya NIM kosong), dan hasilnya API mengembalikan status 400 Bad Request dengan pesan error yang sesuai, bukan error teknis PHP karena tetap memanfaatkan validasi dari MahasiswaService yang sudah dibangun sebelumnya.

## E. KESIMPULAN

Melalui Acara 15 ini, mahasiswa memahami bahwa API memungkinkan aplikasi lain mengakses data tanpa harus menyentuh database secara langsung, dan JSON berperan sebagai format standar dalam pertukaran data tersebut. Dengan memanfaatkan kembali Service Layer yang sudah dibangun pada praktikum sebelumnya, endpoint API yang dibuat tetap konsisten secara validasi dan logika bisnis dengan aplikasi web, sekaligus membuktikan manfaat arsitektur berlapis (Controller–Service–Repository): logika bisnis cukup ditulis sekali, lalu dapat dipakai ulang oleh berbagai jenis "pintu masuk" baik web biasa maupun API.

